

Webcam Gaze Detection System

Estimating where a user is looking on their screen using only a standard laptop webcam — no infrared illuminators, no depth sensor, no head-mounted hardware. A complete technical reference: architecture, algorithms, data formats, API surface, measured performance, and failure modes.

VERSION	DATASET SCHEMA
2026-07-21	v3
PLATFORM	OPERATING SYSTEM
Legion R7000 ARP8	Windows 11
CAMERA	NATIVE RESOLUTION
Realtek integrated	1280 × 720

01 Overview

Problem framing, measured performance, comparison to commercial systems

02 System architecture

Pipeline diagram, module inventory, runtime dependencies

03 Conventions & coordinate systems

Coordinate spaces, scale factors, angle conventions

04 Algorithms

Landmarks, head pose, distance, positioning gate, gaze, calibration, drift, aim assist, fixation, retraining

05 Data formats

CSV schema v3, calibration model, camera focal file

06 API reference

Public functions and classes per module

07 Configuration reference

Every tunable constant and the effect of increasing it

08 Verification methodology

Accuracy grid, classification bands, error topology

09 Limitations & failure modes

Known constraints, mechanisms, mitigation status

10 Operational procedures

Setup, routine use, diagnostics, failure interpretation

A gaze-tracking system that estimates where a user is looking on their screen using only a standard laptop webcam — no infrared illuminators, no depth sensor, no head-mounted hardware.

The system solves three problems in sequence:

- 1 **Where is the face, and where is it in 3D space?** — MediaPipe landmarks + pinhole distance estimation.
- 2 **Which direction are the eyes pointing?** — L2CS-Net gaze model.
- 3 **Which screen pixel does that direction correspond to for this user?** — per-person polynomial calibration.

A positioning gate constrains the user to a fixed region of space so step 3 remains valid, and a self-improving data loop collects labeled training samples during normal use.

Measured performance

MEAN ERROR 2.0–2.4 cm On-screen miss distance	BEST REGION 0.4–1.5 cm Centre and upper screen	WORST REGION 3.5–6.5 cm Corners
FRAME RATE 8–15 fps CPU-bound on gaze inference	CALIBRATION ≈100 s 86 s sweep + 14 s validation	DATA YIELD ≈500 / sweep Plus 100–200 per game

CONTEXT

For reference, commercial webcam-based eye trackers typically achieve 2–4 cm; dedicated IR hardware achieves under 1 cm.

System architecture



Module inventory

Module	Type	Responsibility
gaze_pipeline.py	library	Face detection, square smoothed crop, ONNX gaze inference
gaze_features.py	library	Per-frame extraction of all 32 measured features
positioning_gate.py	library	Distance + centering constraints, parking-sensor overlay
calibration_utils.py	library	Polynomial fit, robust refit, model I/O, dataset I/O
milestone1_face_mesh.py	demo	Landmark visualization
milestone2_eye_headpose.py	demo	Eye crops + head pose
milestone3_gaze_model.py	demo	Raw (uncalibrated) gaze arrow
milestone4_calibration.py	tool	Calibration; also exports shared session primitives
milestone5_positioning.py	demo	Positioning gate + focal calibration
milestone6_test_accuracy.py	tool	Quantitative accuracy measurement
milestone7_gaze_game.py	app	Gamified test, data collection, aim assist, retraining
milestone8_live_cursor.py	app	Live gaze cursor, optional OS mouse control
inspect_dataset.py	tool	Human-readable dataset summary

DEPENDENCY NOTE

Milestones 6, 7 and 8 import session primitives (`FrameReader` , `phase_a_positioning` , `measure_drift` , `CANVAS_W/H`) from `milestone4_calibration.py` . Milestone 4 is therefore a de-facto shared runtime module, not merely a script.

Conventions & coordinate systems

Space	Definition
Frame	Camera pixels, origin top-left, 1280×720, horizontally mirrored
Landmark	MediaPipe normalized [0,1] relative to frame
Canvas	Fixed 1280×720 render surface, stretched to display by OpenCV fullscreen
Screen-normalized	$(x, y) \in [0,1]^2$, origin top-left — the calibration output space
Physical	Centimetres, from Windows <code>GetDeviceCaps(HORZSIZE/VERTSIZE)</code>

Canvas ↔ physical scale (development display, 34.4 × 19.4 cm): 37.2 px/cm horizontally, 37.1 px/cm vertically — near-identical, so pixel radii translate to centimetres uniformly.

Angles: gaze pitch/yaw are in radians everywhere after decoding. Positive yaw = subject looking to their right in the mirrored view; positive pitch = looking up. Head pose is in degrees.

4.1 Face and iris landmarks

MediaPipe FaceLandmarker (Tasks API, `face_landmarker.task`) in `RunningMode.VIDEO`, `num_faces=1`, detection/tracking confidence 0.5. Video mode requires monotonically increasing timestamps; the pipeline synthesizes them as `frame_idx × (1000/30) ms`.

Index	Anatomy	Consumer
468	Left iris center	Distance (IPD), iris features
473	Right iris center	Distance (IPD), iris features
1	Nose tip	Centering check
33, 133, 362, 263	Eye corners	Iris ratio, EAR
234, 454, 10, 152	Face extents	Face width / height
0–477	All	Gaze-model crop bbox

CAPTURE RESOLUTION IS FUNCTIONALLY SIGNIFICANT

The sensor is natively 16:9 at 1280×720. Requesting 640×480 causes the driver to center-crop to 4:3, narrowing the effective horizontal field of view. This invalidates the assumed focal length (biasing all distance readings closer than reality) and halves the pixel count across the iris.

4.2 Head pose

Enabled via `output_facial_transformation_matrixes=True`. The 4×4 matrix is computed by MediaPipe from its full internal 3D face model. Rotation extracted with SciPy:

```
rotation = np.array(transformation_matrix)[:3, :3]
pitch, yaw, roll = Rotation.from_matrix(rotation).as_euler('xyz', degrees=True)
tx, ty, tz = np.array(transformation_matrix)[:3, 3]
```

RATIONALE

A hand-rolled 6-point `solvePnP` was replaced because exact anatomical 2D↔3D correspondence is easy to get subtly wrong on a mirrored frame, which produced errors on the order of 60° yaw / 170° roll while the subject faced forward.

4.3 Distance estimation (monocular, known-size)

The same principle a camera-only vehicle uses to judge distance: project an object of known real-world size and measure its apparent size. Our known object is the interpupillary distance (IPD).

```
distance_cm = focal_px · REAL_IPD_CM / ipd_px
ipd_px = ||(landmark473 - landmark468) ⊙ (W, H)||2
```

Focal length resolution order:

- 1 **Measured (preferred)** — `calibrate_focal()` at a known distance: `focal = ipd_px · known_dist_cm / REAL_IPD_CM`, persisted to `models/camera_focal.json`, linearly rescaled if capture width changes. Measured value on this machine: 1073.4 px.
- 2 **Estimated fallback** — `focal = W / (2 · tan(HFOV/2))`. At $W = 1280$, $HFOV = 60^\circ \rightarrow 1108.5$ px.

ipd_px	90	110	130	150
distance_cm (fallback focal)	77.6	63.5	53.7	46.6

Error sources

Source	Magnitude	Mitigation
Individual IPD variation (± 0.4 cm on 6.3)	$\approx \pm 6\%$ bias	Wide tolerance band; per-user focal calibration
Unpublished true camera FOV	Systematic scale	c-key focal calibration at measured 50 cm
Head yaw foreshortening IPD	Reads farther	Centering constraint limits extreme yaw
Landmark jitter	$\pm 1\text{--}2$ px $\rightarrow \pm 1$ cm	Band is 20 cm wide; not threshold-sensitive

Absolute accuracy is deliberately secondary — the gate enforces a band, and any systematic bias shifts that band consistently for a given user.

4.4 Positioning gate

Constant	Value	Meaning
MIN_DIST_CM / MAX_DIST_CM	45 / 65	Allowed distance band
WARN_MARGIN_CM	5	Yellow zone inside each green edge
CENTER_TOL	0.12	Max nose deviation, fraction of frame
REAL_IPD_CM	6.3	Assumed pupil separation
ASSUMED_HFOV_DEG	60	Fallback FOV

```
in_zone = (MIN ≤ d ≤ MAX) ∧ (|dx| ≤ TOL) ∧ (|dy| ≤ TOL)
RED = ¬in_zone
YELLOW = in_zone ∧ (near a distance edge v |offset| > 0.7·TOL)
GREEN = otherwise
```

`evaluate(landmarks, frame_shape)` returns a dict with `distance_cm`, signed `dx/dy`, `distance_ok`, `centered`, `in_zone`, `zone`, and guidance messages. Returns `None` when IPD is degenerate (< 1 px).

Overlay. Parking-sensor display: discrete level segments spanning 30–80 cm in 5 cm steps (nearest at bottom), allowed band drawn green with yellow warn margins, active segment highlighted; a centre-tolerance box with nose marker; directional guidance; and a border flash whose frequency scales $2 \rightarrow 8$ Hz with the magnitude of violation.

ENFORCEMENT CONTRACT

Every consumer discards gaze data while `in_zone` is false. This is the mechanism by which the constraint buys accuracy — out-of-position samples are never recorded, so they cannot contaminate the fit.

4.5 Gaze estimation

- **Model:** L2CS-Net trained on Gaze360, exported to ONNX, executed on `CPUExecutionProvider` with `intra_op_num_threads = 4`.
- **Preprocessing:** BGR→RGB, resize 448×448, scale to [0,1], ImageNet mean/std normalization, HWC→CHW, batch dimension.
- **Decoding:** two 90-bin logit vectors → softmax → expectation over bin indices → $\text{deg} = \sum(p_i \cdot i) \cdot 4 - 180$ → radians.

CRITICAL IMPLEMENTATION DETAIL

The ONNX graph emits `(yaw_bins, pitch_bins)` — **not** `(pitch, yaw)`. Handled in `gaze_pipeline._predict_gaze_radians()`. Reversing it silently transposes the gaze axes and was a genuine historical defect.

Crop stabilization. The bounding box over all landmarks is converted to a square (side = max extent × 1.3) and smoothed across frames: $b_t = \alpha \cdot b_{\text{measured}} + (1-\alpha) \cdot b_{t-1}$, $\alpha = 0.3$. Squareness preserves aspect ratio through the 448×448 resize; smoothing prevents crop jitter appearing as gaze jitter. State is module-level and must be cleared via `reset_bbox_smoothing()`.

4.6 Calibration

Phase A — positioning gate

Blocks until `zone == "GREEN"` is held continuously for `HOLD_GREEN_SECS = 2.0`; any RED/YELLOW frame resets the timer.

Phase B — smooth-pursuit sweep

A dot traverses a polyline at constant speed while samples are recorded every frame. Path geometry (`build_pursuit_path`), margin $m = 0.05$:

- **Snake:** 5 horizontal rows at $y \in \text{linspace}(m, 1-m, 5)$, alternating direction.
- **Perimeter:** right edge ascent → top edge → left edge descent → bottom edge.
- **Diagonal:** bottom-right → top-left through the centre.

Total: 15 vertices, path length 10.27 screen-units, duration $10.27 / 0.12 = 86$ s.

DESIGN RATIONALE

A pure snake only touches each edge instantaneously at a turn, and the settle-time filter then discards the samples immediately following that turn — leaving corners with effectively no training data, so the polynomial extrapolated there. This produced reproducible corner errors up to 6.2 cm. The perimeter pass gives every edge a dedicated slow traversal approached from two directions; measured top-right error fell from 6.2 cm to 0.7 cm.

Sample admission

Filter	Threshold	Reason
Positioning gate in-zone	in_zone	Constraint enforcement
Time since direction change	≥ 0.3 s	Eye lags the dot after a turn
Face detected, crop non-empty	—	Validity

Fit. `PolynomialFeatures(degree=3)` over the 2 gaze features yields 10 terms —

`[1, p, y, p2, py, y2, p3, p2y, py2, y3]` — feeding two independent `LinearRegression` models (x and y).

Robust refit. Fit → residual norms → retain samples within `median + 2.5 · 1.4826 · MAD` → refit on survivors (only if ≥ 60 remain). The 1.4826 factor converts MAD to a normal-consistent σ estimate. Removes blinks and glances away.

WHY DEGREE 3

Accuracy degrades when looking downward because the eyelid occludes the pupil, a nonlinearity a quadratic cannot represent. Raising the degree reduced worst-corner error from 8.4 cm to 4.1 cm. With 400+ samples and 10 terms, overfitting risk is low.

Rest interval

A 5 s pause precedes validation. Validating immediately after 86 s of continuous pursuit measurably inflated error (18.3% vs 9.8% of screen) due to ocular fatigue.

Phase C — held-out validation

5 static points (centre + 4 mid-quadrants), each 1.2 s dwell + 1.5 s sampling, median-aggregated. Absent from the training path, so the resulting error is a genuine generalization estimate — unlike `fit_error_normalized`, which is training-set error and optimistic.

4.7 Drift correction

Problem. The mapping is conditioned on the exact posture held during calibration. A 1–2° difference in head pitch at a later session displaces every prediction by a near-constant offset — diagnosable in the accuracy report as error vectors that are all parallel and equal in length.

```
drift = predict(median_pitch, median_yaw) - (0.5, 0.5)
corrected(x, y) = clamp0.1( predict(x, y) - drift )
```

Run at the start of milestones 6/7/8 by sampling gaze at a single known centre point. This is the standard procedure used by commercial eye trackers.

CAVEAT

Because drift is measured at screen centre, the centre test point in milestone 6 becomes partially self-fulfilling — evaluate quality primarily from edge and corner points.

4.8 Aim assist

Zone	Radius	Physical	Behaviour
Snap	≤ 130 px	3.5 cm	<code>pull = 1.0</code> — cursor locks to target centre
Magnet	≤ 330 px	8.9 cm	<code>pull = (330 - d) / (330 - 130)</code> , linear fade
None	> 330 px	—	Raw cursor

`assisted = raw + (target - raw) * pull`. The visible target ring is only 70 px (1.9 cm); the magnet field is deliberately not rendered (revealable with `d`), so assistance is imperceptible to an observer.

SIZING CONSTRAINT

`ASSIST_RADIUS` must exceed the model's typical error (4–6 cm $\approx$ 150–220 px) or the attractor is never entered. The original implementation used 160 px with a 65% maximum pull — it could neither reach the cursor nor fully land it, which is why it appeared non-functional.

4.9 Fixation detection & data integrity

Threat model. Aim assist moves the cursor onto the target regardless of where the user is actually looking. Deriving training labels from the assisted cursor would let the assist manufacture its own ground truth, poisoning the dataset.

Countermeasure. Labeling consults raw gaze only. A sample is admitted when `is_fixating()` holds:

Condition	Threshold	Purpose
≥ 3 samples spanning the hold window	0.25 s	Sufficient evidence at 8–15 fps
Spatial spread	≤ 120 px (3.2 cm)	Distinguishes fixation from saccade
Centroid distance to target	≤ 420 px (11.3 cm)	Rejects looking elsewhere entirely

WHY STABILITY RATHER THAN PROXIMITY

Gating on closeness to the target would admit only samples where the model is already correct, from which nothing can be learned. Rows where the model is currently wrong but the eye is demonstrably fixating carry the corrective signal.

4.10 Incremental retraining

- 1 Append the session's fixation samples to `data/gaze_samples.csv`.
- 2 Load the complete accumulated dataset.
- 3 If `len` ≥ 200 , refit with `robust_fit_samples` at the existing degree.
- 4 Carry forward `validation_error` and `position_constraints`; record `trained_on` source counts; persist.

KNOWN CEILING

Pooling sessions without session-aware features degraded training error to 17.7% once eight sessions were mixed: the same `(gaze_pitch, gaze_yaw)` legitimately maps to different screen points across postures, and a model that cannot see which session a row came from averages contradictory evidence. Resolving this requires position-aware features — the reason the 32-column schema exists.

Data formats

5.1 data/gaze_samples.csv — schema v3, 39 columns

Column order is chosen for a human opening the file in a spreadsheet:

1-3	source, user, session_id	what this row is
4-5	target_x, target_y	THE ANSWER (ground truth)
6-37	the 32 measured features	what the camera saw
38-39	timestamp, schema_version	when / layout version

Group A — MEASURED features (columns 6-37, "user data")

Prefix	Columns	Units	Signal
gaze_	gaze_pitch, gaze_yaw	radians	L2CS-Net output — strongest single predictor
head_	head_pitch/yaw/roll	degrees	Head rotation from transformation matrix
head_	head_tx/ty/tz	matrix units	Head translation (tz ≈ camera distance)
iris_	iris_{left,right}_{x,y}	normalized	Absolute iris position
iris_	iris_*_ratio_{x,y}	ratio	Iris position within the eye box — head-invariant eye direction
eye_	eye_{left,right}_{inner,outer}_{x,y}	normalized	Eye corner geometry
ear_	ear_left, ear_right	ratio	Eye aspect ratio: ~0.3 open, ~0.1 closed
face_	face_width, face_height, ipd_px	norm / px	Scale cues covarying with distance
pos_	pos_distance_cm, pos_face_dx/dy	cm / fraction	Gate-derived position and framing

Group B — LABEL (columns 4-5, "actual data")

`target_x`, `target_y` — float [0,1], the on-screen stimulus the user was looking at. Named `target_*` deliberately: this is the coordinate the program rendered, never a model output. No prediction is stored anywhere in this file.

Group C — META (columns 1-3 and 38-39)

Column	Purpose
source	milestone4_pursuit milestone7_game — named for the producing script so the CSV is self-describing
user	Identity, from <code>GAZE_USER</code> env var (default "default")
session_id	Per-process ID, YYYYMMDD-HHMMSS — enables session-aware training

Column	Purpose
timestamp	ISO-8601 batch write time
schema_version	Layout the row was written under (1, 2 or 3)

WHY SESSION_ID MATTERS

Head posture differs between sittings, so the same gaze angles legitimately map to different screen points across sessions. Without this column a pooled fit averages contradictory evidence. It permits per-session normalization, session-held-out validation, and drift modelling.

Derived feature definitions

Iris ratio. Raw iris pixels confound eyeball rotation with head position. The ratio projects the iris onto the outer→inner corner axis:

```
x_ratio = ((iris - outer) · (inner - outer)) / ||inner - outer||2
y_ratio = (iris_y - midline_y) / ||inner - outer||
```

giving 0 at the outer corner and 1 at the inner corner regardless of where the face sits in frame.

EAR. Standard 6-point formula $(\|p_2 - p_6\| + \|p_3 - p_5\|) / (2\|p_1 - p_4\|)$ over landmarks (33, 160, 158, 133, 153, 144) left and (362, 385, 387, 263, 373, 380) right.

Schema migration

`_migrate_dataset()` runs on every write and handles three cases in place:

- **v1 (6-col) / v2 (9-col) → v3** — remaps pitch→gaze_pitch, yaw→gaze_yaw, x→target_x, y→target_y, distance_cm→pos_distance_cm; tags rows `user="legacy"`.
- **v3 with old column order → current order** — same columns, reordered.
- **Short source labels → script-named labels.**

Unknown layouts are left untouched rather than risk corruption.

SELF-DOCUMENTATION

`write_data_dictionary()` regenerates `data/README.md` alongside the CSV on every write, so a plain-English explanation of every column always sits next to the data and cannot go stale. `inspect_dataset.py` prints a runtime summary: row counts by source, users, sessions, feature coverage, and a 3×3 screen-coverage grid.

5.2 models/calibration_model.pkl

```
{
  "poly":      PolynomialFeatures,    # fitted transformer
  "reg_x":     LinearRegression,      # screen-x regressor
  "reg_y":     LinearRegression,      # screen-y regressor
  "degree":    int,
  "fit_error_normalized": float,      # training error (optimistic)
  "validation_error": float|None,    # held-out error (authoritative)
  "position_constraints": {"min_dist_cm", "max_dist_cm", "center_tol"},
  "trained_on": dict,                # {source: count}, after retraining
}
```

5.3 models/camera_focal.json

```
{"focal_px": 1073.4275140253867, "frame_w": 1280}
```

gaze_pipeline

Function	Signature	Returns
detect_face	(frame, landmarker, frame_idx)	(landmarks, bbox, transform_matrix) or (None, None, None)
predict_gaze_from_crop	(face_crop_bgr)	(pitch_rad, yaw_rad)
get_gaze_reading	(frame, landmarker, frame_idx)	(pitch, yaw, bbox)
get_smoothed_square_crop	(frame, landmarks, padding_ratio=0.3)	(crop, bbox)
reset_bbox_smoothing	()	— clears module state

gaze_features

Function	Signature	Returns
extract_features	(landmarks, frame_shape, gaze_pitch, gaze_yaw, transform_matrix, gate_status)	dict of all 32 features
eye_aspect_ratio	(landmarks, pts, w, h)	float EAR
iris_ratio	(landmarks, iris_idx, outer_idx, inner_idx, w, h)	(x_ratio, y_ratio)
head_pose_from_matrix	(matrix)	(pitch, yaw, roll, tx, ty, tz)
FEATURE_COLUMNS	constant	Canonical 32-column order

positioning_gate.PositioningGate

Method	Signature	Returns
evaluate	(landmarks, frame_shape)	status dict or None
draw_overlay	(frame, status)	annotated frame (in place)
focal_px	(frame_w)	float
calibrate_focal	(ipd_px, frame_w, known_dist_cm=50.0)	float; persists JSON

calibration_utils

Function	Signature	Notes
fit_calibration	(samples, degree=2)	Callers pass degree=3
robust_fit_samples	(samples, degree=3, mad_factor=2.5, min_keep=60)	→ (model, n_dropped)
apply_calibration	(model, pitch, yaw)	→ (x, y) clamped to [0,1]

Function	Signature	Notes
append_dataset	(samples, source, path, user, session_id)	Feature dicts + target_x/y; auto-migrates
load_dataset	(path)	→ 4-tuples for the current 2-feature fit
load_dataset_full	(path)	→ list of dicts, all 39 columns
write_data_dictionary	(path)	Regenerates data/README.md
save_calibration / load_calibration	(model[, path])	Pickle I/O

milestone4_calibration — shared session primitives

Symbol	Purpose
FrameReader(cap, gate)	<code>.read(with_gaze=True, with_features=False)</code> → {status, pitch, yaw, features}
phase_a_positioning(reader, window)	Blocking gate; True on success
measure_drift(reader, window, model)	→ (dx, dy) or None
CANVAS_W, CANVAS_H	Canvas dimensions (1280 × 720)

Configuration reference

Constant	Module	Default	Effect of increasing
MIN_DIST_CM / MAX_DIST_CM	positioning_gate	45 / 65	Wider band: easier to stay in zone, weaker constraint
WARN_MARGIN_CM	positioning_gate	5	Earlier yellow warning
CENTER_TOL	positioning_gate	0.12	Looser centering
REAL_IPD_CM	positioning_gate	6.3	Scales all distances proportionally
ASSUMED_HFOV_DEG	positioning_gate	60	Larger → shorter reported distances
BBOX_SMOOTHING	gaze_pipeline	0.3	More responsive, more jitter
PURSUIT_SPEED	milestone4	0.12	Faster sweep, fewer samples
PURSUIT_ROWS / PURSUIT_MARGIN	milestone4	5 / 0.05	Denser rows / edge clearance
SETTLE_TIME	milestone4	0.3	Stricter post-turn rejection
POLY_DEGREE	milestone4	3	More flexible; overfitting risk if data sparse
OUTLIER_MAD_FACTOR	milestone4	2.5	More permissive outlier retention
HOLD_GREEN_SECS	milestone4	2.0	Longer stability requirement
TARGET_RADIUS	milestone7	70 px	Larger visible hit ring
ASSIST_RADIUS / SNAP_RADIUS	milestone7	330 / 130	Stronger, wider assistance
FIXATION_MIN_HOLD	milestone7	0.25 s	Stricter fixation evidence
FIXATION_STABILITY_PX	milestone7	120 px	More permissive stability
FIXATION_MAX_RAW_DIST	milestone7	420 px	Admits samples further from target
RETRAIN_MIN_SAMPLES	milestone7	200	Later first retrain
GAZE_SMOOTHING	milestone7	0.25	More responsive cursor, more jitter
TRAIL_SECONDS	milestone8	1.5	Longer cursor trail

Verification methodology

`milestone6_test_accuracy.py` measures a 3×3 grid at 5% / 50% / 95% of each axis — including true corners, the hardest extrapolation region. Points are disjoint from both the training path and Phase C validation points.

Per point: 1.2 s dwell (discarded) + 1.5 s sampling (median-aggregated) → drift-corrected prediction → Euclidean error converted to centimetres via the Windows-reported physical display size.

Mean error	Classification
< 1.5 cm	Excellent — approaching commercial systems
< 3.0 cm	Good — reliable for button-sized targets
< 5.5 cm	Fair — regions and quadrants only
≥ 5.5 cm	Poor — recalibration required

ERROR TOPOLOGY

The report additionally diagnoses error topology: edge-dominant error indicates insufficient polynomial flexibility or edge coverage; centre-dominant error indicates postural change since calibration; uniformly parallel error vectors indicate session drift.

Limitations & failure modes

Limitation	Mechanism	Mitigation	Status
Posture-dependent accuracy	Mapping conditioned on calibration pose	Positioning gate + drift correction	Mitigated, not eliminated
Downward gaze degradation	Eyelid occludes pupil	Degree-3 polynomial	Partially mitigated
Corner extrapolation	Sparse edge training data	Perimeter sweep pass	Resolved (6.2 → 0.7 cm)
Per-user IPD variance	Fixed 6.3 cm assumption	Optional focal calibration	Bounded ($\sim \pm 6\%$)
CPU-bound throughput	ONNX inference dominates frame time	EMA smoothing masks latency	8-15 fps; ~ 0.25 s cursor lag
Fixed 1280×720 canvas	Relies on OpenCV fullscreen stretching	Per-machine CANVAS_W/H override	Environment-dependent
Model expressiveness ceiling	Fit consumes only (pitch, yaw)	Position-aware features now collected	Schema ready; model pending
Single-user model	Calibration is per person	Retrain per user	By design

KNOWN ENVIRONMENT ISSUE

On systems whose OpenCV build does not stretch fullscreen windows, the 1280×720 canvas renders centred at native size and stimulus dots do not reach the display edges. Remedy: set `CANVAS_W`, `CANVAS_H` to that machine's resolution. Note that rendering at high resolutions (e.g. 2560×1440) measurably reduces frame rate and therefore sample yield.

Operational procedures

Initial setup — per user, per machine

```
$env:GAZE_USER = "arsi"           # tag rows with the operator
python milestone5_positioning.py   # verify tracking; 'c' at a measured 50 cm
python milestone4_calibration.py   # ~100 s; writes calibration_model.pkl
python milestone6_test_accuracy.py # establish baseline accuracy
```

Routine use

```
python milestone7_gaze_game.py     # collects data, retrains automatically
python milestone8_live_cursor.py   # 'm' toggles OS mouse control
python milestone6_test_accuracy.py # re-measure periodically
python inspect_dataset.py          # what is in the dataset right now
```

Diagnostics

```
# Dataset composition
python -c "from calibration_utils import load_dataset; s,c = load_dataset(); print(len(s), c)"

# Saved model metadata
python -c "from calibration_utils import load_calibration; m = load_calibration(); \
print({k: m[k] for k in m if k not in ('poly','reg_x','reg_y')})"
```

Interpreting failures

Symptom	Probable cause	Action
All errors parallel and equal	Session drift	Automatic correction; recalibrate if severe
Corners bad, centre good	Insufficient edge data / degree too low	Recalibrate; verify perimeter pass runs
Centre bad, edges good	Posture changed since calibration	Recalibrate in the intended working posture
Errors large and random	Poor lighting or tracking loss	Improve illumination; check milestone 1 stability
Distance reading implausible	Wrong focal length or capture resolution	Re-run focal calibration; confirm 1280×720
Assist appears inactive	Cursor never enters magnet radius	Verify ASSIST_RADIUS exceeds current mean error